

區塊鏈作業一：離散對數問題求解

學號：B1128019

姓名：黃教丞

使用算法：Pollard's Rho Algorithm

問題說明

這次作業要解的是離散對數問題（Discrete Logarithm Problem, DLP）。簡單來說就是給定質數 p 、生成元 g 、目標值 h 和子群階 n ，要求解出 x 使得 $g^x \equiv h \pmod{p}$ ，其中 $0 \leq x < n$ 。

DLP 是很多密碼系統的基礎，像是 Diffie-Hellman 和 ElGamal。對大數來說暴力破解基本上不可行，所以需要比較聰明的方法。我用的是 Pollard's Rho 算法，時間複雜度是 $O(\sqrt{n})$ ，比暴力的 $O(n)$ 快很多。

Pollard's Rho 算法原理

Pollard's Rho 是個機率算法，主要想法是利用生日悖論（birthday paradox）來找碰撞。

基本流程是這樣：

1. 從隨機起點開始在群裡面做確定性的隨機遊走
2. 追蹤每個點 $x = g^a \cdot h^b \pmod{p}$ 的係數 (a, b)
3. 當兩條不同的路徑走到同一個點就產生碰撞
4. 利用碰撞關係就可以算出 x

數學上的推導：假設找到碰撞點 $x_1 = x_2$ ，也就是：

$$1 \quad g^{a_1} \cdot h^{b_1} \equiv g^{a_2} \cdot h^{b_2} \pmod{p}$$

整理之後可以得到：

$$1 \quad g^{(a_1 - a_2)} \equiv h^{(b_2 - b_1)} \pmod{p}$$

因為 $h = g^x$ ，所以：

$$1 \quad x \equiv (a_1 - a_2)(b_2 - b_1)^{-1} \pmod{n}$$

R-adding 方法

為了建立隨機遊走，我用的是 R-adding 方法。先預先計算 64 個 multiplier： $M_i = g^{(\delta_{ai})} \cdot h^{(\delta_{bi})} \pmod{p}$ ，然後根據當前的 x 值選分區（ $i = x \pmod{64}$ ），每步更新就是：

- x 乘上 M_i
- a 加上 δ_{ai}
- b 加上 δ_{bi}

都要 $\pmod{p}$ 相對應的模數。

Distinguished Points

為了省記憶體，我用了 distinguished points (DP) 的技巧。不是每個點都存，只存那些符合特定條件的點（例如最高的 d 個 bits 是 0）。這樣記憶體用量可以從 $O(\sqrt{n})$ 降到 $O(\sqrt{n} / 2^d)$ 。實作上我根據問題大小選擇 DP bits：

- 小問題 ($n < 2^{30}$) 用 8-10 bits
- 中等問題 ($2^{30} \leq n < 2^{60}$) 用 12 bits
- 大問題 ($n \geq 2^{60}$) 用 14 bits

實作方式

我主要寫了兩個版本：

CPU 單線程版本 (pollard_simple_cpu.py)

這版用 Python 寫，直接用 Python 內建的大整數運算。適合解 order 小於 2^{65} 的問題。核心邏輯很簡單：

```

1  while not found:
2      # 隨機起點
3      a = random(1, ORDER)
4      b = random(1, ORDER)
5      x = pow(G, a, P) * pow(H, b, P) % P
6
7      # 走到 distinguished point
8      while not is_distinguished(x):
9          partition = x % N_PARTITIONS
10         x = x * multipliers[partition]['factor'] % P
11         a = (a + multipliers[partition]['delta_a']) % ORDER
12         b = (b + multipliers[partition]['delta_b']) % ORDER
13
14     # 檢查有沒有碰撞
15     if x in dp_table:
16         solve_collision()
17     else:
18         dp_table[x] = (a, b)

```

CPU 多進程版本 (pollard_parallel_cpu.py)

Group 8 太大了單線程要跑很久，所以另外寫了多進程版本。用 14 個核心跑（留 2 個給系統），實測速度可以到 2.79 M iter/s，Group 8 大概 34 分鐘可以跑完。

GPU 版本

本來想寫 CUDA 版本加速，也做了完整的 kernel 框架（128-bit 算術、Barrett reduction、constant memory 等等），基礎測試都有過。但是遇到一個問題：要在 GPU 上做 77-bit 的模運算需要完整的 $128 \times 128 \rightarrow 256$ bit 乘法，實作起來比想像中複雜。Barrett reduction 的部分也需要更精確的實現。

測試結果發現模運算算出來的值不對，導致 distinguished point 檢測失敗（DP 數量一直是 0）。考慮到 CPU 版本已經夠快，時間有限，就先把重心放在把 CPU 版本優化好。GPU 版本的架構都做好了，之後有時間可以繼續修。

幾個關鍵的優化

1. 預計算 **multipliers**：64 個 multiplier 都先算好存起來，避免重複計算
2. **Distinguished points**：大幅減少記憶體使用
3. 用 **Python 內建的 pow**：pow(base, exp, mod) 比自己寫快很多

解題結果

總共解了 8 個問題（Group 0-7），Group 8 用多進程版本跑了 34 分鐘解出來。

Group 0-4 都很小，基本上瞬間或幾秒就解決了：

- Group 0: p=11, n=10, 不到 0.01 秒, x=6

- Group 1: $p=101$, $n=100$, 5 秒, $x=36$
- Group 2: $p \approx 2^{25}$, $n \approx 2^{24}$, 0.1 秒, $x=4514533$
- Group 3: $p \approx 2^{36}$, $n \approx 2^{35}$, 0.2 秒, $x=9745654779$
- Group 4: $p \approx 2^{50}$, $n \approx 2^{45}$, 6 秒, $x=9437198869499$

Group 5-7 開始要跑比較久：

- Group 5: $p \approx 2^{56}$, $n \approx 2^{52}$, 107 秒（約 2 分鐘）, $x=1207917758047227$
- Group 6: $p \approx 2^{58}$, $n \approx 2^{55}$, 348 秒（約 6 分鐘）, $x=9663351154205691$
- Group 7: $p \approx 2^{70}$, $n \approx 2^{61}$, 851 秒（約 14 分鐘）, $x=618454431123563515$

Group 8 最大：

- Group 8: $p \approx 2^{69}$, $n \approx 2^{65}$, 2061 秒（34.3 分鐘, 用 14 核心）, $x=18202507756056165329$

每個答案都有驗證過，確認 $g^x \bmod p = h$ 且 $x < n$ 。

JSON 格式答案

```
1  {"grp":0,"p":11,"n":10,"g":2,"h":9,"id":"B1128019","cmd":"res","x":6}
2  {"grp":1,"p":101,"n":100,"g":2,"h":78,"id":"B1128019","cmd":"res","x":36}
3  {"grp":2,"p":28524863,"n":14262431,"g":13008203,"h":21592976,"id":"B1128019","cmd":"res"}
4  {"grp":3,"p":58002118547,"n":29001059273,"g":13513236970,"h":30682140414,"id":"B1128019"}
5  {"grp":4,"p":1070407397926837,"n":29733538831301,"g":11391220849310,"h":625095105019029}
6  {"grp":5,"p":7135355843721399,"n":2744367532450823,"g":22779585910668843,"h":631039190}
7  {"grp":6,"p":241767725068081991,"n":24176772506808199,"g":282475249,"h":102203467303750}
8  {"grp":7,"p":1077984309859658267861,"n":1738684370741384303,"g":657139733149567003766,"h"}
9  {"grp":8,"p":404859789103686130573,"n":33738315758640510881,"g":18483589004489576762,"h"}
```

效能分析

CPU 單線程版本的速度大概穩定在 1.4-1.5 M iter/s（百萬次迭代每秒）。Group 3-7 實際跑的迭代次數大概是理論值 $\sqrt{n}$ 的 1-3 倍，算是正常範圍。Group 0 和 1 因為太小所以隨機性影響比較大。

記憶體使用很省，以 Group 7 為例只用了 79,104 個 DP，大概 2.5 MB。總記憶體不到 10 MB（不算 Python 本身的開銷）。

多進程版本用 14 核心可以達到 2.79 M iter/s，比單線程快接近 2 倍（理論上應該更快但因為進程間溝通有 overhead）。

跟暴力搜索比起來，Pollard's Rho 快超多。舉例來說 Group 7 如果暴力搜要 1.7 quintillion 次操作，Pollard's Rho 只要 1.2B 次，快了 140 萬倍。

遇到的一些問題

參數理解錯誤

一開始以為 order 就是 $p-1$ ，後來發現題目給的 n 才是真正的 order。解決方法就是每次都驗證 $g^n \bmod p$ 和 $h^n \bmod p$ 是不是等於 1。

大整數處理

NumPy 的 `random.randint()` 不能處理超過 `uint64` 的數字，但我們的 ORDER 有些會超過。解法是改用 Python 內建的 `random.randint()`，因為 Python 的整數是任意精度的。

GPU 模運算

原本想在 GPU 上做完整的 77-bit 模運算，但發現實作比想像中複雜。Barrett reduction 需要完整的 $128 \times 128 \rightarrow 256$

bit 乘法才能做得準確。基礎測試（GPU 初始化、atomic 操作等）都有過，但模運算的部分還需要更多工作。考慮到時間和 CPU 版本已經夠快，就先暫停 GPU 版本的開發。

Distinguished Point 檢測

一開始 DP 檢測邏輯寫錯，用的是 `if (x_lo <= mask)` 結果會誤判很多點。正確的寫法應該是檢查最高的 d 個 bits 是不是 0： `if ((x_lo >> (64 - dist_bits)) == 0)`。

程式碼 review

中間有用 Claude 和 ChatGPT 做 code review，發現了幾個問題：

在 `pollard_gpu.py` 裡面 multiplier 的計算是錯的：

```
1  # 錯誤寫法
2  muls_factor_lo = np.array([(i * 1000 + 1) % P] for i in range(N_PARTITIONS))
3
4  # 正確寫法
5  factor = (pow(G, delta_a, P) * pow(H, delta_b, P)) % P
```

在 `pollard_gpu_v2.py` 係數更新也有問題：

```
1  # 錯誤：直接加 idx 跟 multiplier 無關
2  a = (a + idx) % order_approx
3
4  # 正確：要加對應的 delta_a
5  a = (a + delta_a[partition]) % ORDER
```

還有 128-bit 算術的部分實作不完整，沒辦法正確處理 56+ bit 的模數。

一些想法

Pollard's Rho 真的蠻有效率的，空間複雜度是 $O(1)$ （用 DP 的話），時間複雜度 $O(\sqrt{n})$ 。跟 Baby-step Giant-step 比起來雖然時間複雜度一樣但空間省很多。

這次實作學到蠻多東西：

1. 大整數的處理要小心，特別是跨語言的時候（Python vs NumPy vs CUDA）
2. 隨機算法的實作要注意細節，一個小地方錯了結果可能差很多
3. 多進程的溝通成本其實不小，實際加速比沒辦法達到理論值
4. GPU 程式設計不是單純把 CPU 程式碼移植過去就好，很多細節要重新考慮

如果之後有時間，可以試著：

1. 把 GPU 版本的 Barrett reduction 實作完整
2. 試試看用更好的 partition function
3. 或許可以根據問題大小自動調整參數（DP bits、partition 數量等）

參考資料

1. Pollard, J. M. (1978). "Monte Carlo methods for index computation (mod p)". *Mathematics of Computation*, 32(143), 918-924.
2. van Oorschot, P. C., & Wiener, M. J. (1999). "Parallel collision search with cryptanalytic applications". *Journal of Cryptology*, 12(1), 1-28.
3. Teske, E. (2001). "On random walks for Pollard's rho method". *Mathematics of Computation*, 70(234), 809-825.

4. Sutherland, A. V. (2007). "Order computations in generic groups". Massachusetts Institute of Technology.